

Deploy Info

1. 사용한 JVM, 웹서버, WAS 등의 버전 및 설정

- **JVM:** OpenJDK 17
 - **WAS (Web Application Server):** Spring Boot v3.5.5 (내장 Apache Tomcat 사용)
 - **웹 서버 (Reverse Proxy):** Nginx (Docker Image: `nginx:latest`)
 - **데이터베이스:** PostgreSQL 14 (Docker Image: `postgres:14`)
 - **메시지 브로커:** Mosquitto 2.0 (Docker Image: `eclipse-mosquitto:2.0`)
 - **인메모리 데이터 저장소:** Redis 6.2 (Docker Image: `redis:6.2-alpine`)
 - **CI/CD 서버:** Jenkins (Docker Image: `jenkins/jenkins:lts-jdk17`)
 - **IDE:** IntelliJ IDEA 2023.3.8 Ultimate Edition
-

2. 빌드 및 실행 시 사용되는 환경 변수

프로젝트 루트 경로의 `.env` 파일에 아래의 환경 변수들이 정의되어 있어야 합니다.

• PostgreSQL 접속 정보

- `POSTGRES_HOST=postgres`
- `POSTGRES_PORT=5432`
- `POSTGRES_USER=c206`
- `POSTGRES_PASSWORD=pw1004`
- `POSTGRES_DB=licar_db`

• Redis 접속 정보

- `REDIS_HOST=redis`
- `REDIS_PORT=6379`

• MQTT 접속 정보

- `MQTT_HOST=mosquitto`
- `MQTT_PORT=1883`

- `MQTT_CLIENT_ID=lifecarrier-spring`
- `MQTT_USER=c206`
- `MQTT_PASSWORD=pw1004`

참고: CI/CD 환경 구동 시 jenkins/.env 파일의 DOCKER_GID 변수도 필요합니다.

```
# DinD
DOCKER_GID=999
```

3. 배포 시 특이사항

1. 초기 SSL 인증서 발급: 최초 배포 전, Nginx가 HTTPS 서비스를 제공할 수 있도록 `jenkins` 폴더에서 `certbot` 명령어를 실행하여 SSL 인증서를 발급받아야 합니다.
2. 이중 Docker Compose 구조: 시스템은 두 개의 `docker-compose.yml` 파일로 관리됩니다.
 - `jenkins/docker-compose.yml`: Jenkins와 Nginx를 실행하여 CI/CD 및 프록시 환경을 먼저 구성합니다.
 - `docker-compose.yml`: Jenkins 파이프라인을 통해 애플리케이션 스택(app, postgres, redis, mosquitto)을 배포합니다.
3. 자동화된 배포: 모든 빌드 및 배포 과정은 `Jenkinsfile`에 정의된 파이프라인을 통해 자동으로 수행됩니다.
4. 데이터베이스 스키마 관리: `spring.jpa.hibernate.ddl-auto` 설정이 `none`이므로, DB 테이블은 최초 실행 시 `db/01_init.sql` 스크립트를 통해 생성됩니다. 스키마 변경 시에는 마이그레이션 관리가 필요합니다.

4. 수동 배포

- SSL 인증서 설정: `./jenkins/`
 - cert bot 을 통해서 ssl 인증서를 발급.

```
sudo certbot certonly --webroot \
-w ./data/certbot/www \
--config-dir ./data/certbot/conf \
--work-dir ./data/certbot/conf \
```

```
--logs-dir ./data/certbot/conf/logs \  
-d j13c206.p.ssafy.io
```

- **Nginx 및 Jenkins 실행:** `./jenkins/`

```
docker compose up -d
```

- **MQTT 브로커 Spring, PostgreSQL, Redis 및 Mosquitto 실행:** `./`

```
docker compose up -d
```

```
git clone https://lab.ssafy.com/s13-mobility-smarthome-sub1/S13P21C206.  
git
```

```
cd apps\backend\jenkins
```

```
sudo certbot certonly --webroot \  
-w ./data/certbot/www \  
--config-dir ./data/certbot/conf \  
--work-dir ./data/certbot/conf \  
--logs-dir ./data/certbot/conf/logs \  
-d j13c206.p.ssafy.io
```

```
echo 'DOCKER_GID=999' > .env
```

```
docker compose up -d --build
```

```
cd ..
```

```
cp { PostgreSQL, Redis, Mosquitto 환경 변수 파일 } .env
```

```
cd licar
```

```
gradlew && gradlew clean build -x test
```

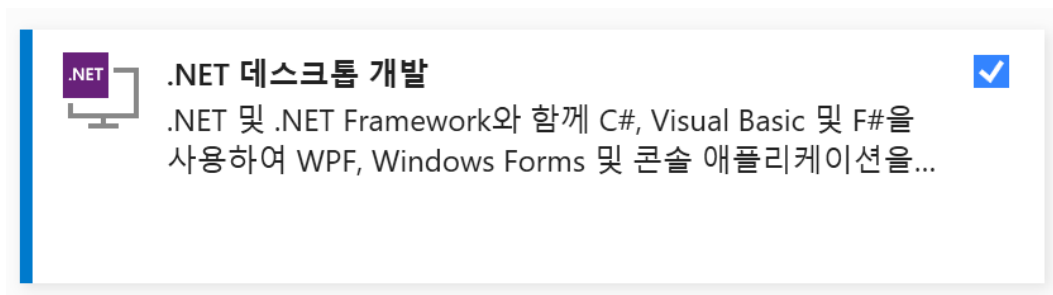
```
docker build -t spring:latest .
```

```
cd ..
```

```
docker compose up -d --build
```

Front End

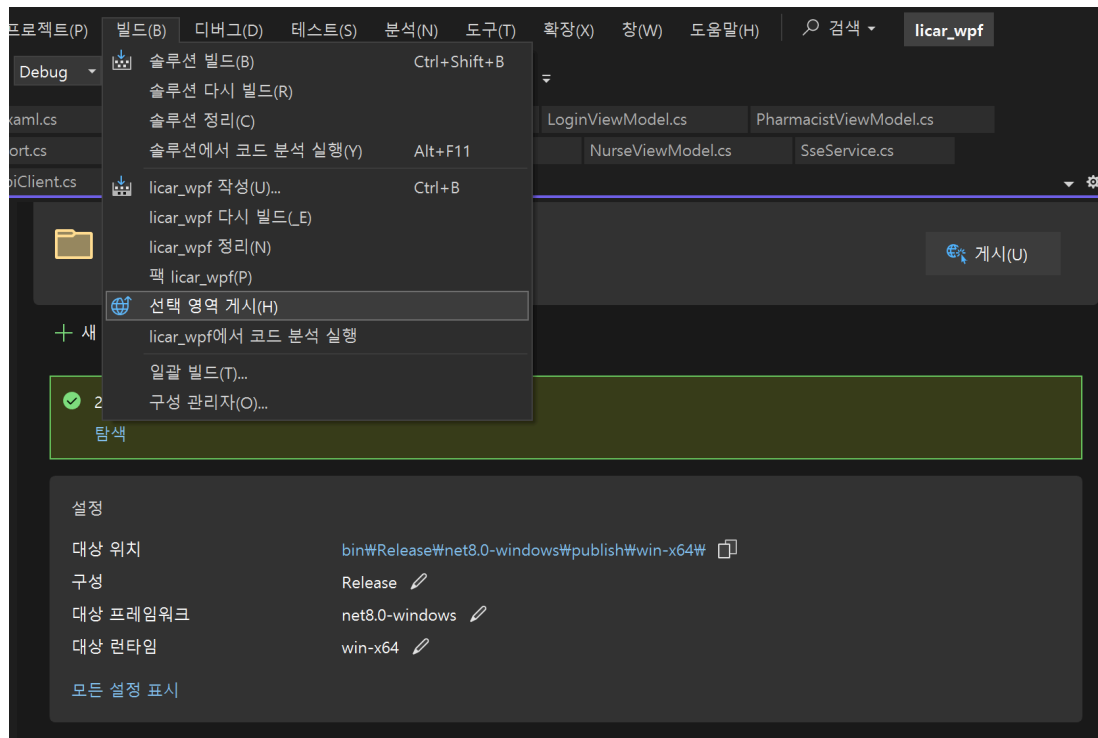
- .NET 8.0 SDK 설치
 - Visual Studio Installer 를 통한 설치



- cmd or PowerShell 의 winget을 통한 설치

```
winget install Microsoft.DotNet.SDK.8
```

- Publish
 - Visual Studio 를 통한 방법



- 우측의 게시 버튼 클릭
- cmd or PowerShell 을 통한 방법

```
dotnet publish licar_wpf.csproj -c Release /p:PublishProfile=FolderP
rofile
```